

Supporting Decisions in a Near-Unpredictable Game

A technical description of ABRA, a decision-support model family for competitive Pokémon

Version 2.4.0 · Last updated 2026-07-23 Will Hooper · ABRA

This is a living document, updated in the same pass as any change to the code, together with the deck and the technical documentation. A prior conclusion is never silently rewritten; new information is added and what changed is stated. See [CHANGELOG.md](#).

Abstract

ABRA is a decision-support model family for **Pokémon Champions VGC, Regulation M-B, best-of-one closed-sheet ladder**. It continuously ingests public battle replays from Pokémon Showdown and stores the durable facts of every game, then builds small, CPU-trainable models on that store. Its central empirical finding governs its design: **predicting the winner of a game from the two team sheets is near-impossible in this format — even a player-Elo model ties a coin**. ABRA therefore does not sell outcome prediction. It follows the recipe that worked in poker, Diplomacy, and sports analytics: *support decisions, don't predict outcomes*, and judge every model by a proper score against an honest baseline with a confidence interval. This paper states the empirical ceiling, the data model, each model with its validated result (including two honest negatives), the mathematics, the limits, and the road to the in-battle engine (ALAKAZAM).

1. The empirical ceiling (why the design is what it is)

On 600+ held-out real Champions games, a Bradley-Terry player-Elo model reaches a held-out log-loss of **0.687 against a coin's 0.693** — a real but negligible edge. A cloned-policy rollout engine (MEDICHAM) does *worse* than a coin as a raw win-predictor (log-loss ≈ 1.2 ; it picks the actual winner on $\sim 44\%$ of decisive calls, i.e. it is systematically *inverted*, over-backing fast offensive teams that lose more). After held-out Platt recalibration it only edges the coin (0.6897 vs 0.6931).

The conclusion is not "our models are weak." It is a property of the game: a two-player, zero-sum, **imperfect-information, simultaneous-move** game with a non-transitive metagame has an irreducible outcome-prediction ceiling from team sheets alone. This is the same reason expected-goals (xG) models in football predict *shot quality* rather than final scores. **Design consequence: stop predicting outcomes; support decisions.** Everything below serves that.

2. Data: store raw, analyse on top

ABRA reads Showdown's public replay API (`search.json?format=` , `search.json?user=` , `<id>.log`); it reads nothing private and creates no accounts (`SECURITY.md`). The extractor (`engine/durable-ingest.js` , `extract()`) turns one battle log into one durable record:

Field	Meaning
<code>id</code> , <code>date</code>	replay id and upload time
<code>p1</code> , <code>p2</code>	<code>{name, rating, bot}</code> per player
<code>six.p1/p2</code>	the revealed team of six
<code>brought.p1/p2</code>	the four actually brought
<code>lead.p1/p2</code>	the two led
<code>sets</code>	per species, the moves / item / ability the replay <i>revealed</i>
<code>turns</code>	per-turn events (moves, damage, faints, status, field)
<code>winner</code>	the winning name

The store is append-only JSON Lines keyed by replay id: idempotent, deduplicated at read time, and grown hourly by a GitHub Action. The **governing rule** is *store raw, analyse on top*: every filter (rating tier, humans-only, archetype, playstyle) is a re-computation over the store, never a re-pull. Changing how we segment games is free; the fetch is a one-time cost. About 2,600 public games/day are available, and the store grows ~18k/week, so every model below sharpens on its own over time.

3. The validated foundation — exact damage (MEDICHAM)

The one component that is *not* a coin flip is the damage engine. MEDICHAM's Gen-9 doubles damage pipeline (`engine/medicham2-browser.js`) is validated against `@smogon/calc` (the community ground-truth) on 31 meta scenarios: **within 5% on 100% of scenarios, median error 0%, worst 3%** (16-roll rounding). This is gated in CI (`engine/validate_damage.js` → `data/damage-validation.json`). Every model that reasons about damage builds on this, and "will this move KO?" is a *winnable* prediction, unlike "who wins the game."

4. The models and their validated results

Every probability ships a **proper score** (log-loss and/or Brier), a **confidence interval** (clustered by game where states within a game are correlated), and an **honest baseline**, persisted to JSON and gated in CI.

4.1 GURU — meta / matchup matrix (descriptive)

From REAL outcomes, `engine/guru.py` builds a 13-archetype × 13-archetype matchup matrix over 5,199 games, each cell a win-rate with a **Wilson score interval**. GURU is *descriptive*: its own predictive test shows per-game winner prediction from the matrix ties a coin (log-loss 0.7122 vs

0.6931), exactly as §1 predicts. Its value is honest matchup structure with error bars, and it is the real (not simulated) payoff matrix that SLOWKING solves. Output: `data/guru-matchups.json`, `data/guru.js`.

4.2 XATU — opponent belief (modest, useful)

`engine/xatu.py` learns, per species, the set (item/ability/moves) usually run, and predicts the opponent's next move from state. On held-out human moves the behaviour-clone reaches **top-1 35.9% (CI 35.2–36.5)**, **top-3 71.6%**, cross-entropy 2.27 nats — beating a species-agnostic baseline (4.54) and uniform-over-moveset (2.91). A modest but real signal; human move choice has genuine entropy. Output: `data/xatu.json`, `data/xatu.js`; harness `engine/eval_policy.py` → `data/policy-eval.json`.

4.3 PORY — mid-game win-probability value net (the win)

The pivot's proof. `engine/pory.py` reconstructs per-turn board state (mons alive out of four, mean active HP, turn) and fits a logistic value net. Held-out, clustered by game: **log-loss 0.567 vs coin 0.693**, beating a material-sign heuristic, **calibrated to ECE 1.6%**, CI **[0.548, 0.583]**. The *live board is predictable even though the pre-game sheets are not* — the thesis, demonstrated. PORY is wired into KADABRA as a per-turn "you're at X%". Output: `data/pory.js`; report `data/pory-eval.json`.

4.4 CHOMP-EV — do CHOMP's brings beat humans'? (honest NULL)

The winnable team-preview test. For each held-out game (both full sixes, both actual brings, the winner), `engine/chomp_ev.js` ranks each side's *actual* bring among all 15 candidate brings by CHOMP's exact-damage coverage, and asks whether that quality signal tracks who won. On **1,205 games**: CHOMP's bring ranking **does not beat a coin** (held-out log-loss 0.6918 vs 0.6931, CIs overlap), ties an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (sign test 0.512, CI [0.493, 0.535]). It is **robust to forfeits** (0.505; a forfeit is usually a concession from a losing position, and dropping all forfeits does not change the result), and a measured **selection audit** shows the required "all four revealed" filter is a mild bias (eval 6.5 turns / 1280 rating vs 6.08 / 1267 excluded) that, if anything, *favours* CHOMP — making the null conservative. A **belief-weighted** variant (coverage vs the opponent's likely-4) also ties the coin (0.6924). Interpretation: the bring decision sits at the same near-coin ceiling as pre-game prediction; CHOMP's damage math stays validated and useful as a calculator, but "CHOMP builds better brings" is not yet empirically supported. This negative is a guardrail: it stops optimising a bring metric that carries no held-out winning signal (a Goodhart trap). Report `data/chomp-ev.json`; test `tests/test-chomp-ev.js`.

4.5 SLOWKING — team-preview Nash and the playstyle cycle (suggestive)

`engine/slowking_preview.py` solves a matchup matrix to a mixed-strategy equilibrium and grades it by **exploitability** (the worst-case win-edge a best pure counter extracts; lower is better; Nash ≈ 0), against greedy "single best deck" and uniform baselines, with a bootstrap CI that propagates matchup-count uncertainty (Beta resampling). Over GURU's 13 species-archetypes the equilibrium is far less exploitable than uniform (Nash ≈ 0 vs 0.109), but greedy $\approx$ Nash because this meta is currently near-transitive (a dominant deck). A **playstyle** re-analysis (`engine/`

`playstyle.js` classifies each team as TrickRoom / Rain / Sun / Sand / Snow / Setup / PerishTrap / TailwindOffense / FakeOutBalance / Stall / HyperOffense) surfaces a non-transitive cycle — **TrickRoom** → **HyperOffense** → **Sand** → **TrickRoom** — with a point exploitability gap of 0.115 for greedy. **Honest caveat:** each cycle leg rests on only 13–18 games (win rates 62% / 71% / 67%) with 95% CIs that cross 50%, so the cycle is a **suggestive pattern, not a settled fact**; it will sharpen as the store grows. Reports `data/slowking-eval.json`, `data/slowking-playstyle-eval.json`; test `tests/test-slowking.py`.

5. Mathematics

Wilson score interval (used for every matchup rate) for w wins in n games, $z = 1.96$: $(\hat{p} + z^2/2n \pm z \cdot \sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)}) / (1 + z^2/n)$, with $\hat{p} = w/n$. It is well-behaved at small n and near 0/1, unlike the normal approximation.

Value net. Features $x = [\text{alive_diff}, \text{hp_diff}, \text{my_alive}, \text{foe_alive}, \text{turn}/10]$ are standardised by train mean/std; $P(\text{win}) = \sigma(w \cdot z + b)$. Graded by held-out **log-loss** $-(y \cdot \ln p + (1-y) \cdot \ln(1-p))$ and **Brier** $(p-y)^2$; the coin scores $\ln 2 = 0.6931$ and 0.25 respectively.

Equilibrium and exploitability. Each preview is a two-player zero-sum matrix game on an antisymmetric edge matrix $M[i,j] = (p(i>j) - p(j>i))/2$. Regret matching (Hart & Mas-Colell) converges to an ϵ -Nash. For a strategy x , **exploitability** $= -\min_j (x \cdot M[:,j])$ — the worst-case loss to a best response; the Nash value is 0, so a Nash strategy scores ≈ 0 and a predictable single-deck strategy is punished.

Confidence intervals. Because per-turn states within a game are correlated, CIs are **bootstrapped by resampling games** (clustered), not states. Matchup-matrix uncertainty is propagated by **Beta($n \cdot p + 1$, $n \cdot (1-p) + 1$) resampling** of each cell before re-solving.

Future rating math. For any descriptive meta-rating we will use an **intransitivity-capable** class (blade-chest / low-rank bilinear, Chen & Joachims 2016; or Nash-averaging, Balduzzi et al. 2018) and a **Helmholtz–Hodge / HodgeRank** decomposition (Jiang, Lim, Yao & Ye 2011) to split the matchup flow into a transitive ranking plus a cyclic (rock-paper-scissors) component — the correct tool for "which cores beat which" and for quantifying how cyclic the meta really is.

6. Limitations and honest ceilings

1. **The game-winner ceiling is permanent** ($Elo \approx \text{coin}$). SLOWKING/ALAKAZAM are judged on decision quality and self-play/ladder win-rate, never on match-outcome prediction.
2. **Revealed sets are partial** (a mon that never attacked reveals no moves); belief is a lower bound.
3. **Small samples in the meta layer.** Playstyle and core matchups are thin; those results are suggestive until the store grows.
4. **Policy is the residual GIGO.** The damage is validated; the rollout *policy* is behaviour-cloned and over-credits speed control. The learning path (PORY, ALAKAZAM) partly sidesteps this by learning from real outcomes.
5. **Champions rule specifics** (sleep/paralysis edge cases) are flagged, not yet fully modelled.

7. The road to ALAKAZAM

ALAKAZAM is the in-battle capstone, built last on the inputs above. Given a live position it will output the win-%-optimal move (a mixed strategy) and its value by: (1) a **belief** over the opponent's hidden sets (XATU), updated by a Bayesian filter; (2) **depth-limited search** over the validated damage engine, solving each simultaneous turn as a **matrix game** (regret matching — this removes the speed bias that inverted the greedy engine); (3) a **learned value** at the leaves (PORY, grown to an NNUE-style net); (4) **human-anchoring** (KL-regularised to the behaviour-clone) so it stays strong and unexploitable. Inference is light (CPU / Web Worker / WASM); the strongest version needs offline RL on millions of human + self-play games and a rented cloud GPU. It is judged on decision quality and self-play/ladder win-rate with CIs — never on predicting the winner. A self-play data engine (MEW) is the pacing item toward the millions of games that path needs.

8. References

1. Zinkevich et al., *Regret Minimization in Games with Incomplete Information* (CFR), 2007.
2. Lanctot et al., *Monte Carlo Sampling for Regret Minimization* (MCCFR), 2009.
3. Moravčík et al., *DeepStack*, Science 2017. · Brown & Sandholm, *Libratus*, Science 2018.
4. Brown et al., *Combining Deep RL and Search* (ReBeL), NeurIPS 2020. · Schmid et al., *Player of Games*, 2021.
5. Angliss et al., *VGC-Bench*, arXiv 2506.10326, 2025. · UT-Austin-RPL, *Metamon* (offline RL + transformers), arXiv 2504.04395, 2025.
6. Perolat et al., *DeepNash / R-NaD* (Stratego), Science 2022. · Vinyals et al., *AlphaStar*, 2019.
7. Meta FAIR, *CICERO / piKL* (human-regularised RL, Diplomacy), Science 2022.
8. Chen & Joachims, *Modeling Intransitivity in Matchup Data* (blade-chest), WSDM 2016. · Balduzzi et al., *Re-evaluating Evaluation* (Nash-averaging), NeurIPS 2018.
9. Jiang, Lim, Yao & Ye, *Statistical Ranking and Combinatorial Hodge Theory* (HodgeRank), 2011.
10. Wilson, *Probable Inference, the Law of Succession, and Statistical Inference*, JASA 1927.
11. @smogon/calculator — community damage ground-truth. · Pokémon Showdown replay API.

Companion documents. [Slide deck](#) · [Technical documentation](#) · [Model ledger](#) · [Changelog](#)